Complete Python Pandas Tutorial

Pandas (pd) – Beginner to Advanced

Pandas is an open-source Python library used for data manipulation, cleaning, analysis, transformation, reading and writing datasets, handling missing values, and preparing data for Machine Learning.

1. What is Pandas?

Pandas is an open-source Python library used for working with structured and tabular data. It is especially useful in Data Science, Machine Learning and data analysis. Think of Pandas as an advanced Excel tool that works through Python code.

• Data manipulation

• Data analysis

• Data cleaning

• Data transformation

• Reading and writing datasets

• Handling missing values

• Working with large tabular datasets

• Preparing data for Machine Learning

Why do we need Pandas?

Imagine a CSV file containing 100,000 transactions. Pandas can help find total sales, identify duplicate transactions, calculate average transaction value, find missing product IDs, group sales by store and identify top-selling products.

What does pd mean?

```
import pandas as pd

# pd is the conventional alias for the pandas library
print(pd.__version__)
```

Here, pandas is the library name and pd is its short alias. The alias is a Python convention.

2. Installation and environment setup

```
pip install pandas

# Check the installed version
import pandas as pd
print(pd.__version__)

# Create a virtual environment
python -m venv venv

# Linux
source venv/bin/activate

# Windows
venv\\Scripts\\activate

pip install pandas
```

3. Pandas data structures

Series – one-dimensional data

A Series is similar to a single column in Excel. It contains values and an index.

```
import pandas as pd

numbers = pd.Series([10, 20, 30, 40])
print(numbers)

# Output:
```

```
# 0    10
# 1    20
# 2    30
# 3    40
# dtype: int64
```

DataFrame – two-dimensional data

A DataFrame is similar to an Excel spreadsheet with rows and columns.

```
data = {
    "Name": ["John", "Ram", "Krishna"],
    "Age": [25, 30, 28],
    "Salary": [30000, 45000, 50000]
}

df = pd.DataFrame(data)
print(df)
```

Index	Name	Age	Salary
0	John	25	30000
1	Ram	30	45000
2	Krishna	28	50000

Index

An index identifies rows in a Series or DataFrame. By default, Pandas creates an index starting at zero.

4. Creating Pandas DataFrames

A. Using a dictionary

```
data = {
    "Product": ["Laptop", "Mobile", "Tablet"],
    "Price": [50000, 20000, 15000],
    "Quantity": [5, 10, 8]
}
df = pd.DataFrame(data)
```

B. Using a list

```
data = [
    ["Laptop", 50000, 5],
    ["Mobile", 20000, 10],
    ["Tablet", 15000, 8]
]
df = pd.DataFrame(data, columns=["Product", "Price", "Quantity"])
```

C. Creating a Series

```
marks = pd.Series(
    [80, 90, 75],
    index=["Maths", "Science", "English"]
)
```

D. Creating an empty DataFrame

```
df = pd.DataFrame(columns=["Name", "Age", "Salary"])
```

5. Reading and writing data

Format	Read	Write
CSV	pd.read_csv()	df.to_csv()
Excel	pd.read_excel()	df.to_excel()
JSON	pd.read_json()	df.to_json()
Parquet	pd.read_parquet()	df.to_parquet()
SQL	pd.read_sql()	df.to_sql()

Format	Read	Write
Pickle	pd.read_pickle()	df.to_pickle()

```
# Read CSV
df = pd.read_csv("sales.csv")

# Write CSV
df.to_csv("output.csv", index=False)

# Read Excel
df = pd.read_excel("sales.xlsx")

# Write Excel
df.to_excel("output.xlsx", index=False)

# Read JSON
df = pd.read_json("sales.json")

# Write JSON
df.to_json("output.json", orient="records")
```

For Excel support, install openpyxl. For Parquet, install pyarrow. SQL support requires an appropriate database driver.

6. Understanding Pandas data types (dtypes)

Data type	Meaning	Example
int64	Integer	100
float64	Decimal	99.50
object	General Python objects, often strings	Ram
str / string	String data	Hello
bool	Boolean	True / False
datetime64	Date and time	2026-11-07
category	Repeated categorical values	IT
Int64	Nullable integer	100,

```
print(df.dtypes)

df["Age"] = df["Age"].astype("int64")
df["Salary"] = df["Salary"].astype("float64")
df["Name"] = df["Name"].astype("string")

df["Date"] = pd.to_datetime(df["Date"])

df["Salary"] = pd.to_numeric(
    df["Salary"],
    errors="coerce"
)
```

errors='coerce' converts invalid values into missing values.

7. Exploring and understanding datasets

```
data = {
    "Name": ["Ram", "John", "Krishna", "David", "Sam"],
    "Age": [25, 30, 28, 35, 22],
    "Department": ["IT", "HR", "IT", "Sales", "HR"],
    "Salary": [30000, 45000, 50000, 60000, 25000]
}
df = pd.DataFrame(data)
```

Command	Purpose
df.head()	First 5 rows
df.tail()	Last 5 rows

Command	Purpose
df.sample(5)	Random 5 rows
df.shape	Number of rows and columns
df.size	Total number of cells
df.info()	Column types and non-null information
df.describe()	Statistical summary
df.columns	Column names
df.dtypes	Data types
df.isnull().sum()	Missing values per column
df.nunique()	Unique values per column
df.memory_usage(deep=True)	Memory usage

```
print(df.head())
print(df.shape)
print(df.columns)
print(df.info())
print(df.describe())
```

describe() provides count, mean, standard deviation, minimum, quartiles and maximum for numeric columns by default.

8. Selecting rows and columns

Select one or multiple columns

```
df["Name"]                 # Series
df[["Name", "Salary"]]    # DataFrame
```

Using loc – label-based selection

```
df.loc[0]
df.loc[0:2]
df.loc[0:2, ["Name", "Salary"]]
```

Using iloc – position-based selection

```
df.iloc[0]
df.iloc[0:3]
df.iloc[0:3, 0:2]
```

Scalar access

```
df.at[0, "Name"]   # Label-based
df.iat[0, 1]       # Position-based
```

Feature	loc	iloc
Selection	Index labels	Integer positions
Row example	df.loc[0]	df.iloc[0]
Column example	df.loc[:, 'Name']	df.iloc[:, 0]
Slice end	Included	Excluded

9. Filtering data

```
# Single condition
high_salary = df[df["Salary"] > 40000]

# Multiple conditions (AND)
result = df[
    (df["Age"] > 25) &
    (df["Salary"] > 40000)
]

# OR condition
```

```
result = df[
    (df["Department"] == "IT") |
    (df["Department"] == "HR")
]

# isin()
result = df[df["Department"].isin(["IT", "HR"])]

# between()
result = df[df["Salary"].between(30000, 50000)]

# query()
result = df.query("Salary > 40000")
result = df.query("Age > 25 and Salary > 40000")

# Negation
result = df[~df["Department"].isin(["IT", "HR"])]
```

Use & for AND, | for OR and ~ for NOT. Put each condition in parentheses.

10. Adding, updating and deleting data

```
# Add a column
df["Bonus"] = df["Salary"] * 0.10

# Update a column
df["Salary"] = df["Salary"] + 5000

# Add a row
new_employee = pd.DataFrame([{
    "Name": "Peter",
    "Age": 29,
    "Department": "IT",
    "Salary": 55000
}])
df = pd.concat([df, new_employee], ignore_index=True)

# Update a specific value
df.loc[df["Name"] == "Ram", "Salary"] = 40000

# Delete a column
df = df.drop(columns=["Bonus"])

# Delete a row
df = df.drop(index=0)

# Rename a column
df = df.rename(columns={"Name": "EmployeeName"})
```

11. Handling missing data

Real-world datasets often contain missing values, commonly represented by NaN, None, NaT or .

```
# Identify missing values
df.isnull()
df.isnull().sum()
df.isna().sum()

# Remove rows with missing values
df.dropna()

# Remove columns with missing values
df.dropna(axis=1)

# Fill missing values with zero
df.fillna(0)

# Fill missing salary with average
df["Salary"] = df["Salary"].fillna(df["Salary"].mean())

# Fill with median
df["Salary"] = df["Salary"].fillna(df["Salary"].median())

# Fill with most frequent value
df["Department"] = df["Department"].fillna(df["Department"].mode()[0])

# Forward fill and backward fill
df.ffill()
```

```
df.bfill()
```

Do not fill every missing value with zero automatically. Choose a strategy based on what the missing value represents.

12. Removing duplicate records

```
# Check duplicate rows
df.duplicated()

# Count duplicates
df.duplicated().sum()

# Remove duplicates
df.drop_duplicates()

# Remove duplicates based on Name
df.drop_duplicates(subset=["Name"])

# Keep the last occurrence
df.drop_duplicates(subset=["Name"], keep="last")

# Reset index
df = df.reset_index(drop=True)
```

13. Sorting and ranking

```
# Ascending
df.sort_values(by="Salary")

# Descending
df.sort_values(by="Salary", ascending=False)

# Multiple columns
df.sort_values(
    by=["Department", "Salary"],
    ascending=[True, False]
)

# Rank salary from highest to lowest
df["SalaryRank"] = df["Salary"].rank(ascending=False)
```

14. GroupBy – one of the most important concepts

groupby() groups records based on a particular column. It follows three steps: split data into groups, apply calculations, and combine the results.

```
data = {
    "Department": ["IT", "HR", "IT", "HR", "Sales"],
    "Salary": [50000, 30000, 60000, 40000, 35000]
}
df = pd.DataFrame(data)

result = df.groupby("Department")["Salary"].mean()
print(result)

# Other aggregations
df.groupby("Department")["Salary"].sum()
df.groupby("Department")["Salary"].max()
df.groupby("Department")["Salary"].min()
df.groupby("Department")["Salary"].count()
df.groupby("Department")["Salary"].median()

# Multiple aggregations
result = df.groupby("Department").agg({
    "Salary": ["sum", "mean", "max", "min"]
})

# Named aggregation
result = df.groupby("Department").agg(
    total_salary=("Salary", "sum"),
    average_salary=("Salary", "mean"),
    employee_count=("Salary", "count")
)

# Multiple grouping columns
df.groupby(["Department", "Age"])["Salary"].mean()
```

15. Mathematical and statistical operations

Function	Purpose
sum()	Sum
mean()	Average
median()	Middle value
mode()	Most frequent value
min() / max()	Minimum / maximum
std()	Standard deviation
var()	Variance
count()	Non-missing count
quantile()	Percentile / quantile
corr()	Correlation
cov()	Covariance
abs()	Absolute value
round()	Round values
cumsum()	Cumulative sum

```
df["Salary"].sum()
df["Salary"].mean()
df["Salary"].median()
df["Salary"].std()
df["Salary"].quantile(0.75)

# Correlation between numeric columns
df.corr(numeric_only=True)
```

Correlation measures the strength and direction of a linear relationship; it does not by itself prove causation.

16. String operations

```
# Lowercase
df["Name"] = df["Name"].str.lower()

# Uppercase
df["Name"] = df["Name"].str.upper()

# Remove leading and trailing spaces
df["Name"] = df["Name"].str.strip()

# Replace text
df["Name"] = df["Name"].str.replace("ram", "rama", regex=False)

# String length
df["NameLength"] = df["Name"].str.len()

# Check whether text contains a value
df["Email"].str.contains("gmail", na=False)

# Extract email domain
df["Domain"] = df["Email"].str.split("@").str[-1]

# Check starting text
df["Email"].str.startswith("ram", na=False)
```

The .str accessor applies string operations to an entire Series. It is useful for cleaning names, email domains, vendor codes and product descriptions.

17. Date and time operations

```
df["Date"] = pd.to_datetime(df["Date"])
```

```
# Extract components
df["Year"] = df["Date"].dt.year
df["Month"] = df["Date"].dt.month
df["Day"] = df["Date"].dt.day
df["Weekday"] = df["Date"].dt.day_name()

# Filter dates
result = df[df["Date"] >= "2026-02-01"]

# Calculate date difference
df["DaysDifference"] = (
    pd.Timestamp.today() - df["Date"]
).dt.days

# Generate date range
dates = pd.date_range(
    start="2026-01-01",
    end="2026-01-10",
    freq="D"
)

# Monthly resampling (requires a datetime index)
df = df.set_index("Date")
monthly_data = df.resample("ME").sum()
```

Be careful with time zones and confirm whether timestamps represent local time or UTC.

18. Merge, Join and Concatenate

These operations combine DataFrames and are similar to SQL JOIN operations.

```
employees = pd.DataFrame({
    "EmployeeID": [1, 2, 3],
    "Name": ["Ram", "John", "Krishna"]
})

departments = pd.DataFrame({
    "EmployeeID": [1, 2, 4],
    "Department": ["IT", "HR", "Sales"]
})

# Inner join
result = pd.merge(
    employees, departments,
    on="EmployeeID", how="inner"
)

# Other join types
left = pd.merge(employees, departments, on="EmployeeID", how="left")
right = pd.merge(employees, departments, on="EmployeeID", how="right")
outer = pd.merge(employees, departments, on="EmployeeID", how="outer")

# Concatenate vertically
combined = pd.concat([employees, employees], ignore_index=True)

# Join using index
joined = employees.join(
    departments.set_index("EmployeeID"),
    on="EmployeeID",
    rsuffix="_department"
)
```

Join	Meaning
inner	Matching records only
left	All left records and matching right records
right	All right records and matching left records
outer	All records from both tables

19. Reshaping data

Pivot

```
result = df.pivot(
```

```
    index="Name",
    columns="Subject",
    values="Marks"
)
```

Pivot table

```
result = df.pivot_table(
    index="Name",
    columns="Subject",
    values="Marks",
    aggfunc="mean"
)
```

Melt – wide format to long format

```
long_df = pd.melt(
    wide_df,
    id_vars=["Name"],
    var_name="Subject",
    value_name="Marks"
)
```

Explode – list elements into separate rows

```
df = pd.DataFrame({
    "Name": ["Ram", "John"],
    "Skills": [["Python", "SQL"], ["Java", "AWS"]]
})
result = df.explode("Skills")
```

20. Applying custom functions

Apply to a column

```
def calculate_bonus(salary):
    return salary * 0.10

df["Bonus"] = df["Salary"].apply(calculate_bonus)

# Lambda alternative
df["Bonus"] = df["Salary"].apply(lambda salary: salary * 0.10)
```

Map values

```
department_codes = {
    "IT": 101,
    "HR": 102,
    "Sales": 103
}
df["DepartmentCode"] = df["Department"].map(department_codes)
```

Apply to rows

```
df["Total"] = df.apply(
    lambda row: row["Salary"] + row["Bonus"],
    axis=1
)
```

axis=1 processes rows; axis=0 processes columns. Prefer vectorized operations when possible because they are usually faster.

21. NumPy integration with Pandas

```
import numpy as np
import pandas as pd

data = pd.DataFrame({
    "A": [10, 20, 30],
    "B": [5, 10, 15]
})

data["Total"] = np.add(data["A"], data["B"])
np.mean(data["A"])
np.max(data["A"])
np.sqrt(data["A"])
np.where(data["A"] > 15, "High", "Low")
```

```
# DataFrame to NumPy array
array = data.to_numpy()

# NumPy array to DataFrame
df = pd.DataFrame(array)
```

22. Data visualization using Pandas

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.DataFrame({
    "Month": ["Jan", "Feb", "Mar", "Apr"],
    "Sales": [10000, 15000, 12000, 20000]
})

df.plot(x="Month", y="Sales", kind="bar")
plt.show()
```

Plot type	Usage
line	Trends over time
bar	Category comparison
barh	Horizontal comparison
hist	Data distribution
box	Spread and potential outliers
scatter	Relationship between variables
pie	Proportions

```
df.plot(kind="line")
df.plot(kind="bar")
df.plot(kind="hist")
df.plot(kind="box")
df.plot(kind="scatter", x="Age", y="Salary")
```

23. Working with SQL databases

```
# Install SQLAlchemy and the appropriate database driver
# pip install sqlalchemy

import pandas as pd
from sqlalchemy import create_engine

engine = create_engine(
    "postgresql+psycopg://user:password@localhost:5432/mydb"
)

query = "SELECT * FROM employees"
df = pd.read_sql(query, engine)

# Write DataFrame data to a database table
df.to_sql(
    "employees_backup",
    engine,
    if_exists="append",
    index=False
)
```

• fail – raise an error if the table exists.

• replace – drop and recreate the table.

• append – add records to the existing table.

Use secure credential handling and parameterized queries. Be careful with replace because it drops the existing table.

24. Advanced Pandas concepts

Concept	Function / feature	Purpose
MultiIndex	pd.MultiIndex	Multiple index levels
Categorical	astype('category')	Efficient repeated categories
Rolling	rolling()	Moving-window calculations
Expanding	expanding()	Cumulative statistics
Exponential window	ewm()	Weighted moving averages
Crosstab	pd.crosstab()	Frequency tables
Cut	pd.cut()	Divide numbers into intervals
Get dummies	pd.get_dummies()	One-hot encoding
Factorize	pd.factorize()	Convert categories to integer codes
Compare	df.compare()	Compare DataFrames
Eval	df.eval()	Evaluate column expressions

```
# Moving average
df["MovingAverage"] = df["Sales"].rolling(window=3).mean()

# One-hot encoding
encoded = pd.get_dummies(df, columns=["Department"], dtype=int)

# Categorical conversion
df["Department"] = df["Department"].astype("category")

# Frequency table
pd.crosstab(df["Department"], df["Age"])

# Divide numeric values into intervals
df["AgeGroup"] = pd.cut(
    df["Age"],
    bins=[0, 25, 35, 50, 100],
    labels=["Young", "Adult", "Middle", "Senior"]
)
```

25. Pandas and Machine Learning

Pandas helps prepare structured data before passing it to a Machine Learning algorithm.

• Load the dataset.

• Explore the data.

• Handle missing values.

• Remove duplicates.

• Convert data types.

• Encode categorical columns.

• Select features and target.

• Split training and testing data.

• Train a model.

• Evaluate the model.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier

# Load data
df = pd.read_csv("customers.csv")

# Remove duplicates
df = df.drop_duplicates()

# Fill missing age
```

```
df["Age"] = df["Age"].fillna(df["Age"].median())

# Encode a categorical column
df = pd.get_dummies(df, columns=["Gender"], dtype=int)

# Features and target
X = df.drop(columns=["Purchased"])
y = df["Purchased"]

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Train and evaluate
model = RandomForestClassifier(random_state=42)
model.fit(X_train, y_train)

print("Accuracy:", model.score(X_test, y_test))
```

This is a simplified example. For real projects, use Scikit-learn pipelines and fit preprocessing only on training data to avoid data leakage. Categorical and missing values may need additional preprocessing.

26. Handling large datasets and performance optimization

```
# Read only required columns
df = pd.read_csv(
    "large_file.csv",
    usecols=["Name", "Age", "Salary"]
)

# Read limited rows
df = pd.read_csv("large_file.csv", nrows=1000)

# Read in chunks
for chunk in pd.read_csv("large_file.csv", chunksize=10000):
    print(chunk.shape)

# Optimize types where safe
df["Age"] = df["Age"].astype("int32")
df["Department"] = df["Department"].astype("category")

# Inspect memory
df.memory_usage(deep=True)

# Use Parquet
df.to_parquet("data.parquet", index=False)
df = pd.read_parquet("data.parquet")
```

• Select only the columns and rows you need.

• Use appropriate data types.

• Prefer vectorized operations over loops.

• Use chunks when the complete file does not fit comfortably in memory.

• Push filtering and aggregation into a database when appropriate.

• Consider Parquet for repeated analytical workloads.

27. Pandas vs NumPy vs Matplotlib vs Scikit-learn

Library	Primary responsibility	Example
Pandas	Prepare and analyze data	Clean and group sales records
NumPy	Numerical calculations	Matrix operations
Matplotlib	Data visualization	Bar and line charts
Scikit-learn	Machine Learning	Classification and regression

28. Complete Pandas cheat sheet

Category	Important commands
Creation	pd.DataFrame(), pd.Series()
Reading	pd.read_csv(), pd.read_excel()
Writing	df.to_csv(), df.to_excel()
Inspection	df.head(), df.info(), df.describe()
Selection	df.loc[], df.iloc[]
Filtering	isin(), between(), query()
Missing data	isnull(), dropna(), fillna()
Duplicates	duplicated(), drop_duplicates()
Modification	drop(), rename(), insert()
Sorting	sort_values(), sort_index()
Grouping	groupby(), agg(), transform()
Statistics	mean(), median(), std(), corr()
Combining	merge(), join(), concat()
Reshaping	pivot(), pivot_table(), melt()
Text	str.lower(), str.strip(), str.contains()
Dates	to_datetime(), .dt.year, resample()
Visualization	plot()
Performance	memory_usage(), astype(), chunksize

29. Real-world Pandas project – Employee and Sales Analytics

Build a practical analysis system using a sales dataset.

Column	Example
TransactionID	1001
StoreID	572
Product	Laptop
Quantity	2
UnitPrice	50000
TransactionDate	2026-11-01
Department	Electronics

- Read sales CSV files.
- Clean missing values.
- Identify duplicate transactions.
- Calculate total sales.
- Find top-selling products.
- Calculate monthly revenue.
- Group sales by store.
- Merge product and transaction datasets.
- Generate charts.

• Export the final report to Excel.

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("sales.csv")

df["TransactionDate"] = pd.to_datetime(
    df["TransactionDate"], errors="coerce"
)
df["Quantity"] = pd.to_numeric(df["Quantity"], errors="coerce")
df["UnitPrice"] = pd.to_numeric(df["UnitPrice"], errors="coerce")

df = df.drop_duplicates(subset=["TransactionID"])
df["Revenue"] = df["Quantity"] * df["UnitPrice"]

print("Total revenue:", df["Revenue"].sum())

product_summary = (
    df.groupby("Product")
      .agg(
          units_sold=("Quantity", "sum"),
          revenue=("Revenue", "sum")
      )
      .sort_values("revenue", ascending=False)
)

print(product_summary.head(10))

monthly = (
    df.dropna(subset=["TransactionDate"])
      .set_index("TransactionDate")
      .resample("ME")["Revenue"]
      .sum()
)

monthly.plot(kind="line", marker="o")
plt.tight_layout()
plt.show()

df.to_csv("clean_sales.csv", index=False)
product_summary.to_excel("product_summary.xlsx")
```

30. Your Pandas learning roadmap

Stage	Topics	Suggested time
1	Python and Pandas fundamentals	1–2 days
2	Data exploration and selection	2 days
3	Data cleaning	2 days
4	Data transformation	2–3 days
5	Advanced Pandas	3 days
6	Data analysis project	3–5 days
7	Machine Learning preparation	2–3 days

31. Best official learning resources

• Pandas User Guide: https://pandas.pydata.org/docs/user_guide/

• Pandas Getting Started Tutorials: https://pandas.pydata.org/docs/getting_started/

• Pandas API Reference: https://pandas.pydata.org/docs/reference/index.html

32. Practice and next steps

Learn Pandas lesson by lesson. Focus on why a function is used, when to use it and how it works. Practice with real CSV datasets rather than memorizing hundreds of functions.

• Practice creating Series and DataFrames.

- Read a CSV and inspect its structure.
- Filter rows using multiple conditions.
- Clean missing values and duplicates.
- Group data and calculate summaries.
- Merge two datasets using a common key.
- Build the sales-analysis project.